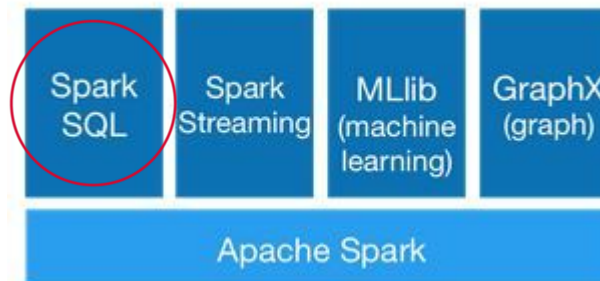


- Introducción
- Conceptos Clave
- Optimizadores
- DataFrames
- UDFs
- SQL
- DataSets
- Ejercicios



Introducción

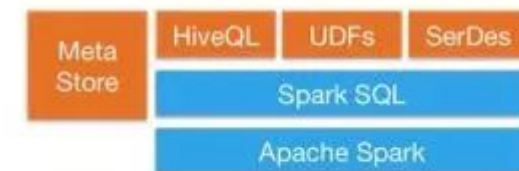
- Es un módulo de Spark para procesar datos estructurados
- Introducido en Spark 1.0
- Conociendo la estructura de los datos, permite hacer optimizaciones
- Apenas requiere conocimiento básico de SQL
- 3 APIs
 - DataFrames
 - SQL
 - Datasets(Spark 2.0)
- Objetivos:
 - Escribir menos código
 - Leer menos información
 - Optimización automatizada
- Dos componentes de optimización
 - Catalyst
 - Tungsten



Conceptos clave

- SQLContext & HiveContext
 - Punto de entrada a la funcionalidad de Spark SQL
 - SQL Context es el genérico
 - Hive Context el específico de Hive
 - Único en cada proceso
- SparkSession
 - A partir de Spark 2.0
 - Primer objeto que se define en un programa Spark
 - Acceso intuitivo a HiveContext para SparkSQL
 - En la spark-shell se llama spark por defecto
 - Permite acceder a sqlContext
 - spark.sqlContext

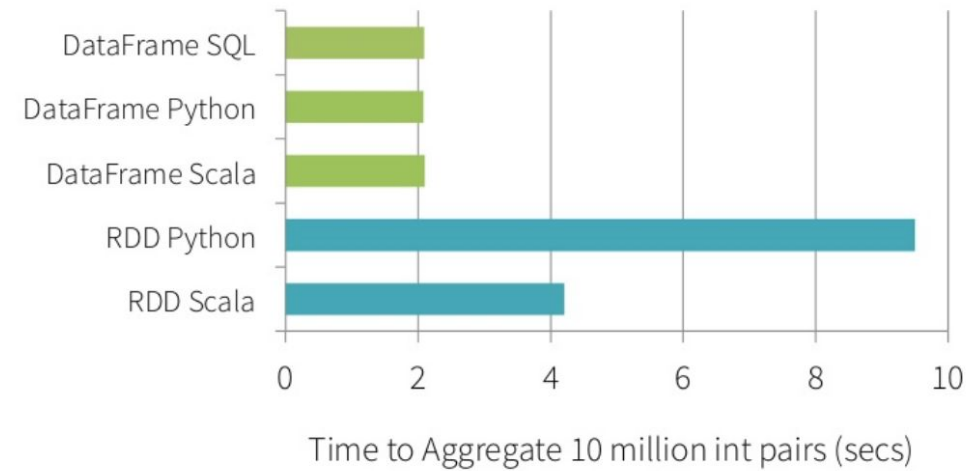
```
val conf: SparkConf = new SparkConf().setAppName("alturasDF")  
    .setMaster("local[+]")  
    .set("spark.hadoop.validateOutputSpecs", "false")  
val sc: SparkContext = new SparkContext(conf)  
val sqlContext = new org.apache.spark.sql.SQLContext(sc)
```



Spark SQL can use existing Hive metastores, SerDes, and UDFs.

DataFrame

- Colección distribuida de Registros
- Inmutable
- Lazy
- Estructurada
 - Registros
 - Columnas con nombre
- Conceptualmente equivalente a una tabla
- Capacidad para inferir el esquema desde la fuente
- Multitud de fuentes de entrada



DataFrame - Creación

- Ficheros estructurados
 - `sqlContext.read.csv / json etc...`
- Tablas Hive
 - `sqlContext.table`
 - `sqlContext.sql`
- BBDD Externas
 - A través de JDBC
- RDDs
 - `rdd.toDF(columns)`
 - `sqlContext.createDataFrame(rdd de rows, schema)`



DataFrame - Reader & Writer

- DataFrameReader
 - Devuelve por sentencia read
 - sqlContext.read.
- Multitud de opciones
 - header
 - inferSchema
 - delimiter
- DataFrameWriter
 - Si no se indica el formato => parquet
 - Admite particionado estático y dinámico
 - partitionedBy
 - Admite escritura en Hive
 - saveAsTable
 - insertInto
 - Permite sobreescritura
 - mode(saveMode.Overwrite)

DataFrame - Esquema

- Todo dataframe tiene un esquema
- Puede obtenerse con el método schema
- Puede imprimirse con printSchema
- Método show muestra DataFrame como si fuera una tabla
- El esquema es un StructType
- Puede crearse a mano
- Cada elemento es un StructField
- Se construye a partir de una secuencia de StructFields

```
scala> df.printSchema
root
 |-- nombre: string (nullable = true)
 |-- edad: integer (nullable = false)
 |-- departamento: string (nullable = true)
```

```
scala> df.show
+-----+-----+-----+
| nombre|edad|  departamento|
+-----+-----+-----+
|Eduardo|  21|administracion|
| Sergio|  25|  desarrollo|
|   Paco|  42| Interiorismo|
+-----+-----+-----+
```

DataFrame - Esquema II

```
scala> val myRdd = sc.parallelize(Seq(("Sergio",33),("Ester",31)))
myRdd: org.apache.spark.rdd.RDD[(String, Int)] = ParallelCollectionRDD[26] at parallelize at <console>:30

scala> val rowsRDD = myRdd.map(r => Row(r._1,r._2))
rowsRDD: org.apache.spark.rdd.RDD[org.apache.spark.sql.Row] = MapPartitionsRDD[27] at map at <console>:31

scala> val schema = StructType(Seq(StructField("name",StringType),
    | StructField("age",IntegerType)))
schema: org.apache.spark.sql.types.StructType = StructType(StructField(name,StringType,true), StructField(age,IntegerType,true))
```

```
scala> val people = spark.createDataFrame(rowsRDD,schema)
people: org.apache.spark.sql.DataFrame = [name: string, age: int]

scala> people.show
+-----+-----+
|  name|age|
+-----+-----+
|Sergio| 33|
| Ester| 31|
+-----+-----+
```


DataFrame - Ejercicio

- Conéctate a la SparkShell
- Acceso al sqlContext a través de:
 - spark.sqlContext
- Crea un RDD con nombre, edad y sexo
- Conviértelo en un DataFrame con el método toDF
- Crea ahora un esquema StructType compuesto de los 3 StructFields
 - Deberás importar:
 - org.apache.spark.sql._
 - org.apache.spark.sql.types._
- Convierte tu RDD original en un RDD de rows y utiliza el método createDataFrame para crear el DF de nuevo

```
scala> res12.printSchema
root
|-- nombre: string (nullable = false)
|-- edad: integer (nullable = false)
|-- dept: string (nullable = false)

scala> df.printSchema
root
|-- nombre: string (nullable = true)
|-- edad: integer (nullable = false)
|-- departamento: string (nullable = true)
```

DataFrame - Transformaciones I

- Acceder a las columnas
 - importando sqlContext.implicits._
 - \$"columnName"
 - importando org.apache.spark.sql.functions._
 - col("columnName")

```
scala> people.select($"age").show
+----+
|age|
+----+
| 33|
| 31|
+----+
```

```
scala> people.select($"name").where("age > 32").show
+-----+
|   name|
+-----+
| Sergio|
+-----+
```

DataFrame - Transformaciones II

- Funciones de agregación
 - Similares a SQL
 - groupBy, avg, sum
 - Se emplean en select o tras un groupBy
 - Método agg permite emplearlas en groupBy
- Funciones sobre columnas
 - substring, length, year, abs, trim
- Selección / filtrado
 - select, where, when

```
scala> people.select(avg($"age"), sum($"age"), count($"age")).show
+-----+-----+-----+
| avg(age) | sum(age) | count(age) |
+-----+-----+-----+
|      32.0 |        64 |           2 |
+-----+-----+-----+
```

```
scala> people.select(length($"age")).show
+-----+
| length(age) |
+-----+
|           2 |
|           2 |
+-----+
```

```
// Entering paste mode (ctrl-D to finish)
people.withColumnRenamed("name", "nombre")
.withColumn("inicial", substring($"nombre", 0, 1))
.drop("age").show
```

```
// Exiting paste mode, now interpreting.
```

```
+-----+-----+
| nombre | inicial |
+-----+-----+
| Sergio | S       |
| Ester  | E       |
+-----+-----+
```

- Añadiendo columnas
 - withColumn
- Renombrando columnas
 - withColumnRenamed, alias
- Eliminando columnas
 - drop

DataFrame - Alturas

- Genera un programa que lea el fichero alturas.csv y calcula la media de altura por sexo
- input a descomprimir:
<https://utad-input.s3.amazonaws.com/rez/meta/data.zip>
subirlo con **docker cp alturasRev.csv namenode:/home**
docker-compose exec namenode bash
hdfs dfs -mkdir -p /user/root/input
hdfs dfs -put /home/alturasRev.csv /user/root/input/
- Filtra los datos erróneos (vacíos o negativos)
 - Utiliza el método where
- Corrige los que vengan mal (en metros en lugar de en centímetros)
- Utiliza el método withColumn y el when

- Agrupa con groupBy
- Emplea el método agg y avg para obtener la media de altura

```
H,180  
M,1.62  
H,176  
M,166  
H,174  
M,-100  
M,158  
M,162  
H,177  
M,163  
H,178  
H,191  
H,173  
H,1.65  
H,176  
M,163  
M,160  
M,157  
M,163  
H,172  
M,170  
H,174  
M,165
```



```
(H,173.46)  
(M,164.17)
```

UDFs

- Son funciones que se aplican sobre columnas
- Definidas por el usuario
- Se definen a partir del método `udf`
 - `import`
`org.apache.spark.sql.functions.udf`
- **Ejercicio:** En lugar de usar el engorroso `when`, modifica el ejercicio anterior para usar una UDF

- Ejemplos
 - `val upperUdf = udf((w:String) => w.toUpperCase)`
 - `val concatProduct = functions.udf((family: String, group: String, tipo: String) => {
 family.myTrim
 .concat(group.myTrim)
 .concat(tipo.myTrim)
})`
 - `val setDealToJoin = functions.udf((sourceSystem: String, product: String) => {

 if (sourceSystem == "MX3_FXFI" && product == "IRDIRSNONE") {
 "Component_UDF_Reference_2"
 } else {
 "deal_id"
 }

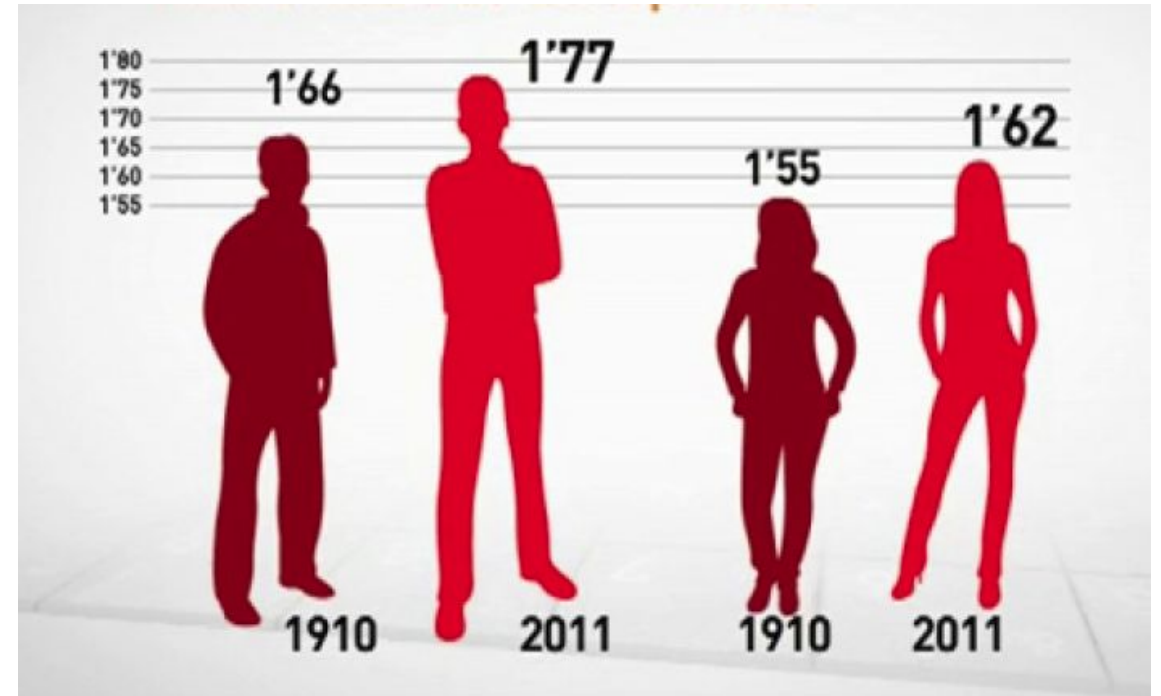
 })`

- Aprovecha la similitud de DataFrame con una tabla
- Permite *publicar* un dataframe como una vista Hive
 - `df.createOrReplaceTempView`
 - `df.createOrReplaceGlobalTempView`
- Permite trabajar con ella empleando queries de HiveQL
- `spark.table`
 - cargar la vista a un DF
- `spark.sql`
 - Query de argumento
 - Cargar un DF resultante de operar con la tabla
- Permite usar UDF
 - `spark.sqlContext.udf.register....`



SQL - Alturas

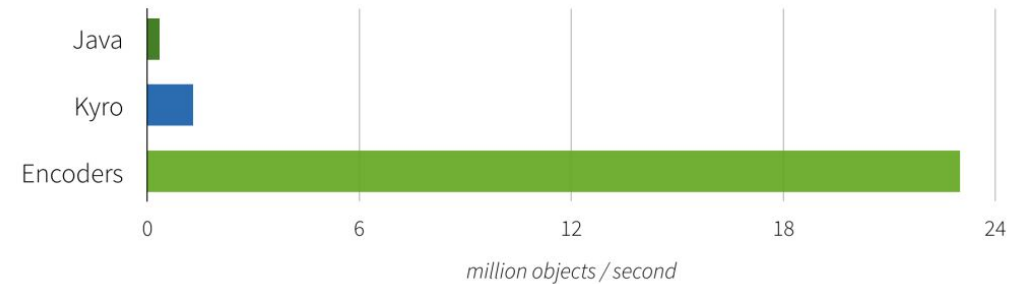
- Modifica o crea un programa nuevo
- Calcula la altura media usando SQL
- Recuerda Registrar la UDF
 - `spark.sqlContext.udf.register....`



DataSets

- Extensión del API de DataFrame
- Tipados
 - Dataset[String]
- Encoder para serialización y deserialización
 - Encoders automáticos para tipos primitivos
 - Automáticos para cases clases (Scala) y Beans (Java)

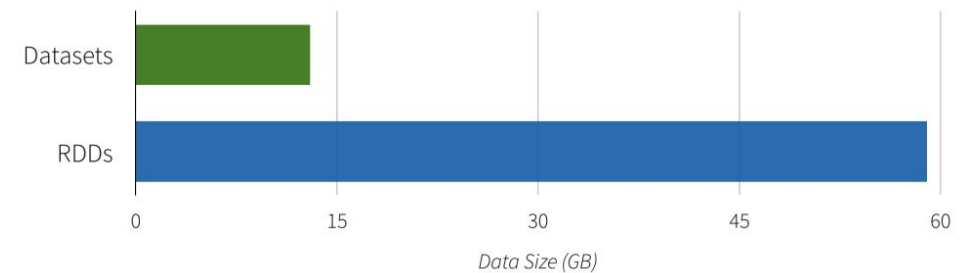
Serialization / Deserialization Performance



Distributed Wordcount



Memory Usage when Caching



DataSets - creación

- Desde un fichero
 - `spark.read.text(inputFile).as[String]`
- Desde un RDD
 - `rdd.toDS`
- Desde un DataFrame
 - (automáticamente)
 - `df.as[Persona]`
 - a mano
 - `df.map{row => Persona(row.getAs[String](0), row.getAs[Int](2))}`



Ejercicio - Películas

- curl <https://utad-input.s3.amazonaws.com/rez/meta/inputs.zip>
-output inputs.zip
- Crea una case class para cada uno de los datasets
- Carga cada uno de los datasets en un DataSet de la clase correspondiente
 - Carga el fichero con spark.read.text
 - Splitea cada fila por el separador
 - Mapea cada fila al objeto que corresponda
- Calcula la nota media que ha realizado cada usuario
- ¿Quién puntúa más alto? ¿ Hombres o mujeres?
- Cuáles son las películas con mayor puntuación



Ejercicio - Productos

- Partiendo de dos datasets
 - Productos
 - id, descripción, precio y stock
 - Transacciones
 - fecha, hora, cliente, producto, número de items, dinero total
- Obtén el cliente que gastase más dinero.
 - Muestra id del cliente y total de dinero
- Obtén un informe de todos los productos vendidos
 - Muestra id, descripción, items vendidos y cantidad total
- Lista los productos que nunca han sido vendidos
 - id, descripción y precio
- Obtén las transacciones de los 10 productos con mayor stock



Ejercicio - Hoteles

- Partiendo de 2 datasets
 - Uno con todos los hoteles de Europa
 - Otro con Ingresos
- Filtrar los Hoteles que pertenezcan a España
 - Mostrar nombre y ciudad
 - El campo cc1 indica el país
- Obtener los ingresos de dichos hoteles
 - Mostrar nombre, ciudad e ingresos
- Obtener por pantalla los 100 hoteles con más ingresos
 - Mostrar nombre, ciudad e ingresos
- Obtener por pantalla las 200 ciudades que más ingresos obtuvieron
 - Mostrar ciudad, e ingresos
- Selecciona los hoteles que no tuvieran ingresos
 - Muestra nombre, ciudad y país



Ejercicio - Netflix

- Partiendo de 2 datasets
 - Uno con las cotizaciones de Netflix
 - Otro las conversiones históricas de USD a EUR
- Se pide
 - Obtén el día que las acciones tuvieran el máximo precio
 - Calcula la media del precio de cierre
 - Calcula el máximo y mínimo de volumen y el día el que se obtuvo cada uno.
 - ¿ Cuántos días el precio de cierre fue mejor de 600\$?
 - ¿ Qué porcentaje de días el precio máximo fue mayor de 500\$?
 - Calcula el precio máximo por año
 - Calcula el precio medio de cierre para cada mes
- Contravaloración
 - Usando el otro dataset, obtén los mismos resultados en EUR
 - Debes aplicar la conversión del día que corresponda
 - Debes aplicar el tipo que corresponda (open, high, low)
 - Para los close no hay tipo de cambio, emplea el open

NETFLIX



¿Preguntas?

GRACIAS